Unit 1 – Using Objects and Methods
Lesson 3: Expressions and Output



Printing to the Screen

- In our IDE, the "console" represents what gets printed to the screen when we run our program
- The program below uses `System.out.print()` and `System.out.println()`
    - `System.out.print()` prints to the console, and then _____
    - `System.out.println()` prints to the console, and then _____

Code	**Console**

`public class Welcome {`
`  public static void main(String[] args) {`
`  System.out.println("Welcome to our class");`
`  System.out.print(5);`
`  System.out.print("=");`
`  System.out.println(2 + 3);`
`  }`
`}`	

Scanner Class

- In programs, we can code our variables to have values, or we can input values from the user and assign them to those variables
- Java uses `System.out` to refer to the _____ device and `System.in` to the _____ device
- By default, the output device is the _____ and the input device is the _____
- You will use the `Scanner` class to create an object to read input from `System.in`

There are three steps to get user input:

1) Import the Scanner class

2) Create the Scanner object

3) Code user input

Import the Scanner Class	Create the Scanner Object	Code User Input
• The `Scanner` class is in the _____ package. • The import statement tells the compiler _____ • We are essentially using code that has already been written and we need to let Java know that we want to import those classes. • Therefore, you will put the following import statement at the top of your program, before your class declaration:	• Syntax for creating a Scanner Object: • `keyboard` is an identifier for your input device • When you want to get user input, you will use the word `keyboard` (or any identifier you make) to do it • You can use `keyboard` as many times as you need throughout your program	• To code user input, first declare a variable that will hold the user inputted value. • `value` is created and then the program waits for user input. • Using `nextInt()` will let the computer wait for the next integer entered and then once "enter" is pressed, that number will be stored in `value`.

Here is a complete program, asking the user for their age and then outputting the result:

Code	Console

```
1: import java.util.Scanner;
2.: public class Main {
3:   public static void main(String[] args) {
4:         Scanner keyboard = new Scanner(System.in);
5:         System.out.print("Enter your age: ");
6:         int age = keyboard.nextInt();
7:         System.out.print("Your age is: " + age);
8:   }
9: }
```

The above program asks for an integer and then uses the method `nextInt()` to get an integer. There are several methods you can use to get the appropriate type from the user:

```
String example = keyboard.next(); //gets the next token (which is a whitespace, space, or enter, by default)
String example2 = keyboard.nextLine(); //gets the entire line up to the enter key being pressed
double example3 = keyboard.nextDouble(); //gets the next decimal entered
```

If your user inputs a value that does not match what the code was expecting (you are asked for an integer but input a String), we'll get an `InputMismatchException`, which is a *run-time error*. That means the code will still compile, but will crash while it is running.

Java's Order of Operations *(Order Precedence)*

1. Parentheses
2. Multiplication, Division, Modulus (left to right)
3. Addition and Subtraction (left to right)

**Note: There is NO built-in exponent symbol (like the carrot symbol, ^)

Code	Console	Reason
`System.out.println(3 + 4 * 2);`		Order of operations is followed and the 4 * 2 is evaluated before the addition.
`int answer = 2 + 5 / 5 * 2;` `System.out.println(answer);`		Between the three operators, division and multiplication have the same precedence, and will be evaluated from left to right, so division happens first and then multiplication, and addition last.

`int x = 9;`
`int y = 2;`
`int z = y + x * x;`
`System.out.println(z);`
		Variables can be used in expressions like numbers. To evaluate, look at the current state of the variables when the expression is reached and evaluate.

`System.out.println(9/2);`		9 and 2 are both integers, so Java will perform _____ This means that the answer is "how many times does 2 fit into 9 evenly?"

`int x = 9;`
`int y = 2;`
`double z = x/y;`
`System.out.println(z);`
		Even though Java knows that z is going to be a double, it's still only looking at x and y, which are both integers, so the expression is evaluated as integer division, and the result is stored in a double variable.

Mixed Expressions

- A mixed expression is an expression that contains operands with _____
- `double` values hold _____ spaces in memory than `int` values, so `double` is considered more complex
- Whenever there's a mixed expression, Java temporarily promotes the less-complex type so that it matches the more-complex type, and then the operator is applied

Code	Console	Reason
`System.out.println(10/4);`		This is integer division, so the result is an integer.
`System.out.println(10.0/4.0);`		This is division between two double values, so the result will be a double.
`System.out.println(10/4.0);`		This is a mixed expression. Java will promote the 10 into a 10.0 and perform division between two double values.

`int a = 10;`
`int b = 4.0;`
`double c = a/b;`
`System.out.println(c);`
`System.out.println(a);`
		This demonstrates how Java will TEMPORARILY promote the less-complex type (int) to evaluate a mixed expression but it does not change it permanently.

`System.out.println(1.0 * 3 / 4);`		The mixed expression 1.0 * 3 is evaluated first, and the 3 is promoted to a 3.0 and 1.0 * 3.0 is 3.0 Then 3.0 /4 is a mixed expression and the 4 is promoted to a 4.0 and 3.0 / 4.0 is double division which will result in a 0.75
`System.out.println("3 + 4");`		A String literal is printed
`System.out.println("3" + 4);`		Mixed expression where the 4 is promoted to a String and string concatenation is performed.

Code	Console	Reason
`System.out.println("3" + 4 + 5);`		This happens because of the way Java allocates memory. Once it starts working with Strings, it continues to look at things at Strings, and interpret '+' as a concatenation operation.
`System.out.println("3" + (4 + 5));`		By putting parentheses around 4 + 5 we indicate to Java that it should do that first. Since 4 and 5 are both ints, it performs a number addition inside the parentheses and then a concatenation to put together the whole statement.

`ArithmeticException`

• Don't divide by 0!
• This causes an `ArithemticException`
• Code will compile correctly, but the program will encounter a run-time error
	`int x = 5;`
	`int y = 0;`
	`int z = x/y;`

Compound Operators

Beyond the arithmetic operations, we also have some *compound operators* that we use in programming:

Regular Operation	`x = x + 5;`	`x = x - 4;`	`x = x * 5;`	`x = x / 6;`	`x = x % 7;`	`x = x + 1;`	`x = x - 1;`
Compound Operation							

Practice: What is the value of the variable after the code is executed?

```
int x = 10;
x += 5;
```

```
int y = 17;
y -= 4;
```

```
int z = 9;
z *= 5;
```

Unary Operators

Pre-Increment	Post-Increment	Pre-Decrement	Post-Decrement
`int g = 5;` `++g;`	`int i = 5;` `i++;`	`int h = 0;` `--h;`	`int p = 0;` `p--;`

- For the code above, it does not matter if you use a pre or post version of the unary operators.
- It does matter when they are parameters in a method (like the `println` method)

Practice: What is printed on each line of code?

```
double example = 4.5;
```

`System.out.println(++example);` Output: _____

`System.out.println(example++);` Output: _____

`System.out.println(example);` Output: _____

Modular Division

- When we divide one integer by another in math class we usually would expect this: _____
- Another way you may have seen this expressed is with a remainder: _____
- _____ is finding the _____ after integer division is performed.
- In Java, the remainder (modulus) operator is _____
- The mod operator has the same precedence as multiplication and division

Practice: Evaluate the expressions below.

10 % 6	39 % 6	12 % 4	17 % 2	0 % 5	5 % 0

Using Mod Operator

- The most popular use of the mod operator is deciding if a number is even or odd
- When mod-ing a number by 2, if the remainder is a _____, the number is even … _____
- When mod-ing a number by 2, if the remainder is a _____, the number is odd … _____

One Last Thing…

- Statements are NOT relationships
- Once the code is run, it does not go back and run again

Code	**Console**

`public class Example {`
`  public static void main(String[] args) {`
`  double bill = 30.0;`
`  double tax = 0.1 * bill;`
 	
`  bill -= 10;`
 	
`  System.out.println("Total Tax: $" + tax);`
`  }`
`}`	